

Les Listes en Python

💡 Définition et Création

Une liste est une collection de données ordonnée et modifiable. C'est l'une des structures de données les plus importantes en Python. Elle est reconnaissable par les crochets `[]` qui l'entourent. On peut y stocker des éléments de types différents (nombres, chaînes de caractères, booléens, etc.).

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Créer une liste vide
>>> fruits = []
>>> # Ajouter des éléments de types différents
>>> fruits.append("pomme")
>>> fruits.append(2)
>>> fruits.append(True)
>>> # Vérifier la longueur d'une liste avec len()
>>> len(fruits)
3
>>> print(f"La liste 'fruits' contient {len(fruits)} éléments.") # La liste 'fruits' contient 3 éléments.
```

La compréhension de liste (List Comprehension)

C'est une syntaxe rapide et très "pythonique" pour créer des listes à partir d'autres collections. Elle combine une boucle et une condition sur une seule ligne. C'est une astuce de Python que vous rencontrerez souvent.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Créer une liste de carrés pairs de 0 à 10
>>> carres_pairs = [x**2 for x in range(0, 11) if x % 2 == 0]
>>> print(carres_pairs) # Affiche [0, 4, 16, 36, 64, 100]
```

⚙️ Modification de la liste

Les listes sont mutables, ce qui signifie qu'on peut ajouter ou supprimer des éléments après leur création.

Ajouter des éléments

- `append(element)`: ajoute un élément à la fin de la liste.
- `insert(index, element)`: insère un élément à un index précis.
- `extend(iterable)`: ajoute tous les éléments d'un autre objet (comme une autre liste) à la fin.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Créer une liste de jours
>>> jours = ["lundi", "mardi", "mercredi", "jeudi", "vendredi", "samedi", "dimanche"]
>>> # Ajouter un jour à la fin
>>> jours.append("jeudi")
>>> # Insérer un jour à un index précis
>>> jours.insert(2, "mardi")
>>> # Ajouter une autre liste à la fin
>>> jours.extend(["mercredi", "jeudi", "vendredi"])
>>> # Vérifier la longueur de la liste
>>> len(jours)
8
>>> print(jours) # ['lundi', 'mardi', 'mercredi', 'jeudi', 'vendredi', 'samedi', 'dimanche']
```

Supprimer des éléments

- `del ma_liste[index]`: supprime l'élément à l'index donné.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Créer une liste de nombres
>>> nombres = [10, 30, 40, 50]
>>> # Supprimer un élément à index 1 (le nombre 30)
>>> del nombres[1]
>>> # Vérifier la longueur de la liste
>>> len(nombres)
3
>>> print(nombres) # [10, 40, 50]
```

- `pop(index)`: supprime l'élément à l'index donné et le renvoie. Si l'index est omis, il supprime et renvoie le dernier élément.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Créer une liste de nombres
>>> nombres = [10, 30, 40, 50]
>>> # Supprimer le dernier élément et le stocker
>>> element_supprime = nombres.pop()
>>> # Vérifier la longueur de la liste
>>> len(nombres)
3
>>> print(nombres) # [10, 30]
```

📄 Copier une liste

Si vous attribuez une liste à une autre variable avec le signe `=`, vous ne faites pas une copie ! Les deux variables pointent vers le même objet en mémoire. On appelle cela une référence.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Créer une liste
>>> liste_a = [1, 2, 3, 4]
>>> # Attribuer la liste à une autre variable
>>> liste_b = liste_a
>>> # Vérifier si c'est une référence ou une copie
>>> liste_b[0] = 5
>>> print(liste_a) # Affiche [5, 2, 3, 4] !
```

Pour faire une vraie copie, on peut utiliser la technique du slicing ou la méthode `.copy()`.

🔍 Indexation et Accès aux éléments

Chaque élément d'une liste possède une position, appelée index. L'index commence à 0.

On peut aussi utiliser des index négatifs pour accéder aux éléments de la liste. L'index `-1` correspond au dernier élément, `-2` à l'avant-dernier, etc.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Créer une liste d'animaux
>>> animaux = ["chat", "oiseau", "poisson"]
>>> # Accéder au premier élément (index 0)
>>> animaux[0]
'chat'
>>> # Accéder au deuxième élément (index 1)
>>> animaux[1]
'oiseau'
>>> # Accéder au troisième élément (index 2)
>>> animaux[2]
'poisson'
>>> # Accéder au dernier élément (index -1)
>>> animaux[-1]
'poisson'
>>> # Accéder à l'avant-dernier élément (index -2)
>>> animaux[-2]
'oiseau'
>>> # Accéder à l'élément à l'index 2 à l'aide de la fonction len()
>>> len(animaux)
3
>>> print(animaux[2:]) # Affiche ['poisson']
```

🔍 Parcourir une liste

Une boucle `for` est l'outil principal pour parcourir les éléments d'une liste.

1. Par élément :

C'est la méthode la plus simple et la plus courante. On parcourt chaque élément de la liste.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Créer une liste de pizzas
>>> pizzas = ["calzone", "pepperoni"]
>>> # Parcourir la liste
>>> for pizza in pizzas:
>>>     print(pizza)
calzone
pepperoni
```

2. Par index :

Utile si vous avez besoin de la position de l'élément en plus de son valeur. On utilise la fonction `range(len(liste))`.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Créer une liste de pizzas
>>> pizzas = ["calzone", "pepperoni"]
>>> # Parcourir la liste par index
>>> for i in range(len(pizzas)):
>>>     print(f"Pizza n°{i} : {pizzas[i]}")
Pizza n°0 : calzone
Pizza n°1 : pepperoni
```

3. Parcours mixte (avec enumerate) :

La méthode la plus élégante pour avoir à la fois l'index et l'élément. On utilise la fonction `enumerate()`.

```
root@ubuntu:~$ python3
Python 3.10.12 shell
>>> # Créer une liste de pizzas
>>> pizzas = ["calzone", "pepperoni"]
>>> # Parcourir la liste avec enumerate
>>> for index, pizza in enumerate(pizzas):
>>>     print(f"Index {index} correspond à la {pizza}.")
Index 0 correspond à la calzone.
Index 1 correspond à la pepperoni.
```

HACK
AGAINST
MACHISM

Fiche d'aide-mémoire - NSI Tani Mallandi

FOLLOW
THE
PYTHONIC
WAY

#ACK AGAINST MACHISM